

자료구조 및 알고리즘

Problem 01

연결 리스트 구현 시 더미 헤드를 사용하는 장단점을 말해보시오.

① 코드가 간결해지고, 일관화 가능.

② 메모리 차지.

③ 연산 속도가 느림
소모된다.

Problem 02

때로는 원소 x 가 존재하는지 체크해주는 메서드 `contains()`도 요구한다. 이것은 본문에서 소개한 메서드 `index(x)`를 호출해서 간단히 해결할 수 있다. 다음 부분을 완성하시오.

메서드 구현 방식 : 문제출제, / pseudo code 식으로

```
def contains(self, x) -> bool:
```

```
    if self.index(x) != -1:
```

```
        return True
```

```
    else:
```

```
        return False
```

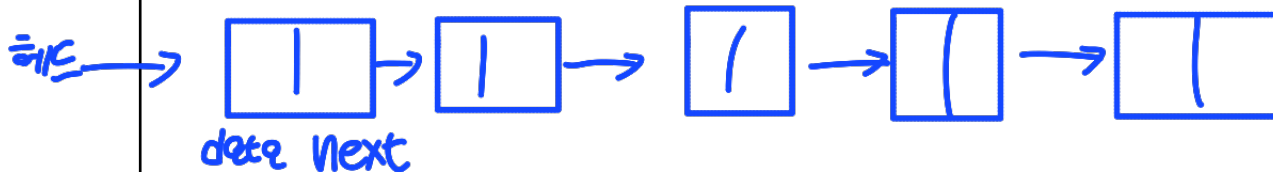
Problem 03

이진 링크드 리스트

3절의 연결 리스트에서 $i \sim j$ 번 원소(첫 원소는 0번 원소라 부른다)를 프린트하는 메서드 `printInterval()`을 작성하시오. [코드 5-15]의 클래스 `LinkedListBasic`에 메서드 `printInterval()`을 더하는 방식으로 하고 다음 부분을 완성하시오.

```
def printInterval(self, i:int, j:int) :
```

그냥 for문 안에서 인덱스가 $[i, j]$ 일때 더하는식 사용.

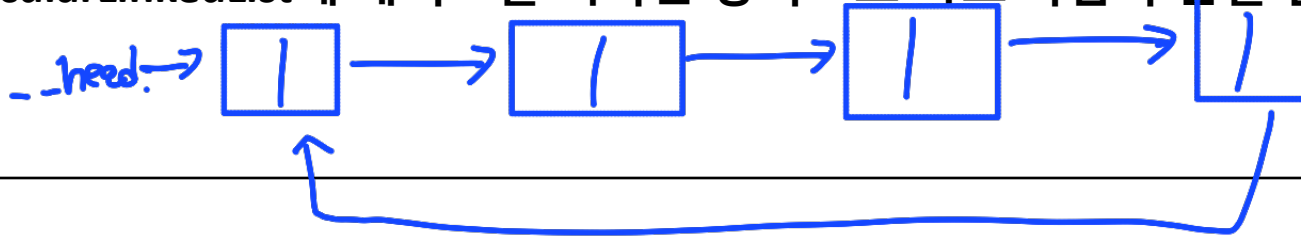


```
curr = self.head.  
for q in range(j-i+1) :  
    curr = curr.next
```

```
if (q < i or q > j) :  
    print(f" {q}번째 원소 {curr.data} ")
```

Problem 04

03번 문제와 똑같은 작업을 [코드 5-22]의 CircularLinkedList에서 하시오. 역시 클래스 CircularLinkedList에 메서드를 더하는 방식으로 하고 다음 부분을 완성하시오.



```
def printInterval(self, i:int, j:int) :
```

```
    index=0  
    for element in self:  
        if index ≥ i and index ≤ j:  
            print(element)  
        if (index==j):  
            break  
        index++
```

Problem 05

본문과 달리, 리스트의 원소 수를 확인하기 위해 변수 `__numItems`를 두지 않고 대신 비현실적이지만 메서드 `numItems()`로 구현했다고 가정한다. 다음 부분에 메서드 `numItems()`를 구현하되 두 가지 방법으로 구현하시오.

1. 재귀를 사용하지 않는 알고리즘으로 구현하시오.

for문. `if (next == None): return`.

2. 재귀 알고리즘으로 구현하시오.

```
def __numItems(t):  
    if (t == None):  
        return 0  
  
    else:  
        return __numItems(t.next) + 1
```

Problem 06

메서드 `pop()`에 인자를 하나 더 주어 `pop(index, k)`와 같이 호출하고, `index`번부터 연속된 `k`개의 원소 (`index`번 원소를 포함하여 `k`개)를 삭제하고자 한다. `pop(index, 1)`을 부르면 원래의 `pop(index)`와 같은 일을 하게 될 것이다. 만일 `index`번 노드부터 시작해서 남은 노드가 `k`개가 안 되면 지울 수 있는 최대 한도인 마지막 노드까지만 지운다. 다음 부분을 채워 넣어 이에 맞게 변형해보시오.

```
def pop(self, index:int, k:int) :
```

```
    prev = self._getNode(i-1)
```

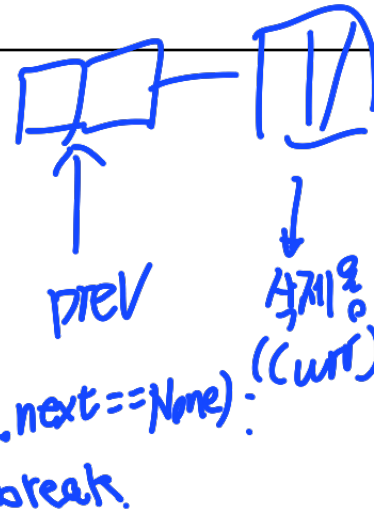
```
    curr = prev
```

```
    for i in range(k):
```

```
        curr = curr.next
```

```
    prev.next = curr.next
```

```
    self._numItems -= 1
```



Problem 07

다음은 5절의 양방향 연결 리스트를 파이썬으로 구현한 코드다. 이 리스트에서는 int 타입의 원소들이 오름차순으로 정렬된 형태로 유지된다. 이 양방향 연결 리스트에 임의의 정수 x를 삽입하는 메서드인 add()를 작성해보시오. 다음 부분을 채워 넣으면 된다.

```
class CircularDoublyLinkedList:
    def __init__(self):
        self.__head = BidirectNode("dummy", None, None)
        self.__head.prev = self.__head
        self.__head.next = self.__head
        self.__numItems = 0
```

...

```
def add(self, x)->None:
```

- ① 첫번째부터 비교.
- ② 넣을 값이 크면 넘김.
- ③ 넣을 값이 작으면 그 자리에 삽입


```
add (self, x):
```

```
    curr = self._head.
```

```
    for i in range(self._numItems):
```

```
        curr = curr.next.
```

```
    if (curr.item == x):
```

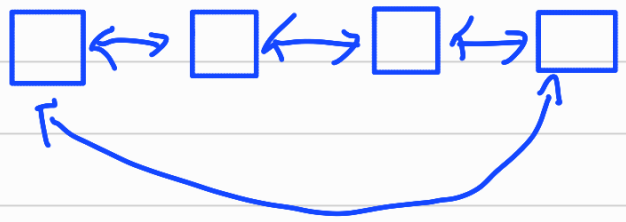
```
        break.
```

```
    newNode = BidirectNode(x, curr.prev, curr)
```

```
    newNode.prev.next = newNode.
```

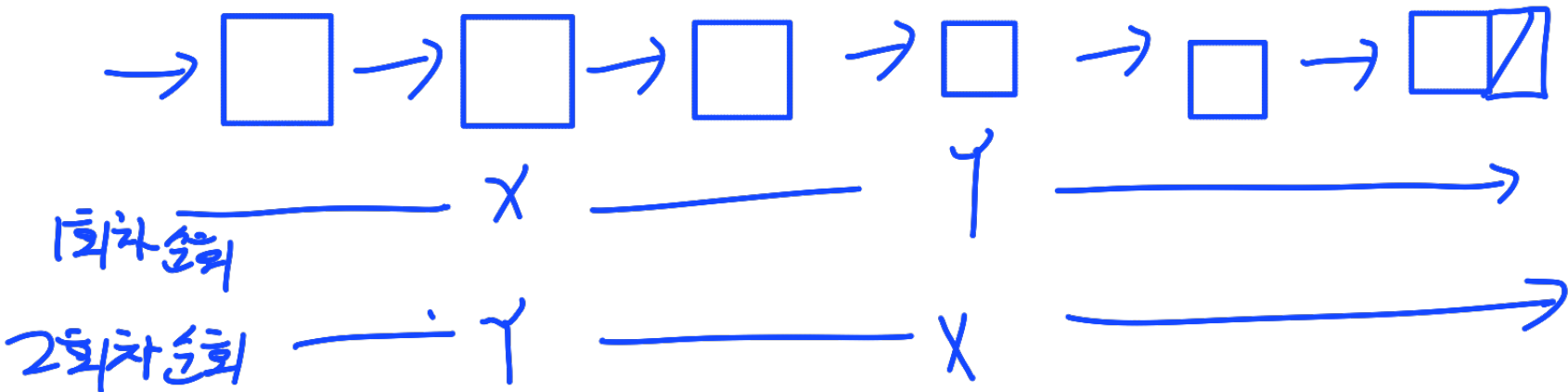
```
    newNode.next.prev = newNode.
```

```
    _numItems += 1
```



Problem 08

두 노드의 레퍼런스 node1과 node2가 있다. 이들은 [코드 5-15]의 클래스 LinkedListBasic의 노드들인데 두 노드가 같은 연결 리스트에 속하는지는 모른다. 이들이 같은 연결 리스트에 속하는지 확인하는 코드를 작성하시오.



Q. 예외

1. 자바 7.7.2

2. 전가위

3. 1.5.0

4. 1.5.0

5. 1.5.0

6. 1.5.0

리스트는 앞에서 체크

Problem 09

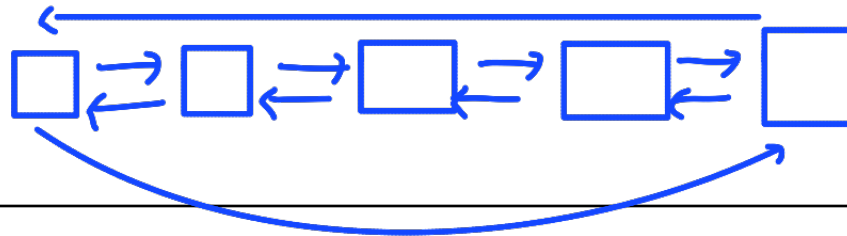
리스트에서 원소 x 를 찾되 맨 마지막에 있는 x 의 위치를 요구하는 메서드 `lastIndexOf(x)`가 필요하다. [코드 5-15]의 클래스 `LinkedListBasic`에 다음 메서드를 추가하시오. 다음 부분을 완성하면 된다.

```
def lastIndexOf(self, x) :  
    lastIndex = -1  
    curr = self._head.  
    for i in range(self._numItems)  
        curr = curr.next  
        if curr.item == x :  
            lastIndex = i  
  
    return lastIndex.
```

Problem 10

Circular에서는 뒤어헤크

09번의 메서드 `lastIndexOf(self, x)`를 [코드 5-25]의 클래스 `CircularDoublyLinkedList`에 추가하시오.



```
def lastIndexOf(self, x) :  
    last index = -1  
    curr = self._head  
    for i in range (self._numItems - 1, -1) :  
        curr = curr.next  
        if curr.item == x :  
            last index = i  
            break  
    return last index.
```

시험에 안낼수준이다.

Problem 11



클래스 LinkedListBasic에 대하여, 리스트를 뒤집는 재귀 메서드를 구현하시오.

Q. 이걸 앞에서 시작하는게 종료조건 좋지않나?

```
def reverse(self) :
```

```
    self.__reverse(self.__head)
```

```
def __reverse(node):
```

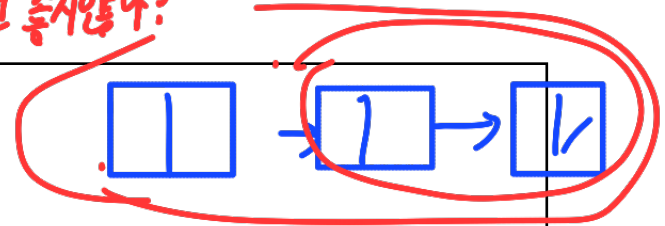
```
    if (node.next is None) : return Node
```

```
    new_head = self.__reverse(node.next)
```

```
    node.next.next = node
```

```
    node.next = None
```

```
    return new_head
```



node

node.next